SNOWFLAKE
Concepts Explained In Detail with Worked SQL Examples

A concept-by-concept study guide -- with runnable SQL for every idea
Understand the WHY, the HOW, and see it in actual code

1. The Three-Layer Architecture
2. Virtual Warehouses & Compute Elasticity
3. Micro-Partitions, Metadata & Pruning
4. Clustering & Automatic Reclustering
5. Caching -- The Three Layers
6. Time Travel & Fail-safe
7. Zero-Copy Cloning
8. Streams -- Change Data Capture
9. Tasks -- Native Orchestration
10. Snowpipe -- Continuous Ingestion
11. Semi-Structured Data & the VARIANT Type
12. Role-Based Access Control (RBAC)
13. Data Masking & Row Access Policies
14. Secure Data Sharing
15. Snowpark -- Code Where the Data Lives
16. Materialized Views & Search Optimization
17. The Credit & Billing Model
18. Table Types: Permanent, Transient, Temporary

The Three-Layer Architecture

Understand this first -- almost every other Snowflake concept is a consequence of this design

1. Storage, Compute, and Cloud Services

Snowflake's single most important architectural idea is that it splits the traditional database into three independent layers instead of bundling storage and compute together on the same machine, which is how most conventional databases worked.

Layer 1: Storage

All table data is stored, compressed and columnar, on the underlying cloud provider's object storage (S3, Blob Storage, or Cloud Storage). You never manage these files directly. Storage is billed purely on compressed volume stored per month, independent of how much you query it.

Layer 2: Compute (Virtual Warehouses)

Queries are executed by Virtual Warehouses -- independently sized, independently billed clusters of compute. Multiple warehouses can read the exact same underlying data without ever copying it or competing with each other for resources.

Layer 3: Cloud Services

This is the coordination brain: authentication, session management, metadata, the query optimizer, and access control (RBAC) enforcement all live here.

■ *THINK OF IT LIKE THIS: Storage is a library's shelves (fixed, contains all the books). A Virtual Warehouse is a reading room you rent by the hour. The Cloud Services layer is the librarian who knows exactly where every book is and checks your ID before letting you in.*

■ **EXAMPLE: Seeing all three layers in one statement**

```
-- Create a warehouse (COMPUTE layer)
CREATE WAREHOUSE etl_wh
  WAREHOUSE_SIZE = 'SMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE;

-- Use it to query data that lives in the STORAGE layer
USE WAREHOUSE etl_wh;

SELECT customer_id, SUM(order_total) AS lifetime_value
FROM   sales.orders                 -- physical data, on cloud storage
GROUP BY customer_id
ORDER BY lifetime_value DESC
LIMIT 10;

-- The CLOUD SERVICES layer silently handled: authenticating you,
-- parsing/optimizing this SQL, and checking your role had SELECT
-- privilege on sales.orders before any compute even ran.
```

Virtual Warehouses & Compute Elasticity

The compute layer, in depth

2. What a Warehouse Actually Is

A Virtual Warehouse is a named cluster of compute nodes, sized from X-Small up to 6X-Large, where each step up roughly doubles both compute power and credit cost per hour.

Auto-suspend and auto-resume

A warehouse can automatically suspend after a period of inactivity so you stop paying credits, and automatically resume the instant a new query arrives.

■ EXAMPLE: Creating a cost-conscious warehouse

```
CREATE WAREHOUSE reporting_wh
  WAREHOUSE_SIZE      = 'X-SMALL'
  AUTO_SUSPEND        = 60          -- suspend after 60 seconds idle
  AUTO_RESUME         = TRUE
  INITIALLY_SUSPENDED = TRUE;

-- Resize it up temporarily for a heavy monthly job, then back down
ALTER WAREHOUSE reporting_wh SET WAREHOUSE_SIZE = 'LARGE';
-- ... run the heavy query ...
ALTER WAREHOUSE reporting_wh SET WAREHOUSE_SIZE = 'X-SMALL';
```

Scaling Up vs Scaling Out

Scaling UP (vertical)

Resizing a single warehouse to a bigger size. Speeds up ONE heavy query (a huge join, a massive sort). Does nothing for many concurrent small queries.

Scaling OUT (horizontal, via multi-cluster warehouses)

Adds more clusters at the same size to absorb concurrent load, controlled by a minimum and maximum cluster count. Doesn't make one query faster -- it lets more queries run at once without queueing.

■ *THINK OF IT LIKE THIS: Scaling UP is hiring one much stronger chef so a single huge order gets cooked faster. Scaling OUT is opening more identical kitchen stations so more separate small orders can be cooked in parallel.*

■ EXAMPLE: A multi-cluster warehouse for a BI dashboard with unpredictable concurrency

```
CREATE WAREHOUSE bi_dashboard_wh
  WAREHOUSE_SIZE = 'MEDIUM'
  MIN_CLUSTER_COUNT = 1
  MAX_CLUSTER_COUNT = 4
  SCALING_POLICY = 'STANDARD'   -- auto-scale out as concurrency rises
  AUTO_SUSPEND = 120
  AUTO_RESUME = TRUE;
```

```
-- Check whether queries are queueing (a scale-OUT signal)
SELECT query_id, warehouse_name, execution_status,
       queued_provisioning_time, queued_overload_time
FROM   TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
WHERE  warehouse_name = 'BI_DASHBOARD_WH'
ORDER BY start_time DESC
LIMIT 20;
```

Micro-Partitions, Metadata & Pruning

The single most important internal concept for performance questions

3. What a Micro-Partition Is

Every table is automatically divided into micro-partitions -- roughly 50 to 500MB each, stored compressed and columnar. Snowflake stores min/max/distinct-count/null-count metadata about each column within each partition.

Pruning -- how a filter actually skips data

When a query filters on a column, the optimizer checks each micro-partition's stored min/max metadata first. If a partition's range can't possibly match, it's skipped entirely -- never read from storage.

■ *THINK OF IT LIKE THIS: Imagine a library where every shelf has a label listing the earliest and latest publication year on it. Looking for a 2026 book, you walk straight past every shelf labeled '1990-2010' without opening it.*

■ EXAMPLE: Checking pruning effectiveness for a query

```
-- A well-filtered query on a date column
SELECT *
FROM   sales.orders
WHERE  order_date = '2026-07-24';

-- Run this right after to inspect how much pruning actually happened
SELECT *
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY())
WHERE query_id = LAST_QUERY_ID();

-- Or, in Snowsight: open the Query Profile for the query and look at
-- the "TableScan" node -> "Partitions scanned" vs "Partitions total".
-- If a highly selective filter still scans close to 100% of
-- partitions, that's a clustering problem (see Section 4).
```

Clustering & Automatic Reclustering

What determines whether pruning actually works well

4. Natural Clustering vs Explicit Clustering Keys

Data loaded together tends to land in the same micro-partitions (natural clustering by load order). This breaks down on huge tables frequently filtered on a column NOT correlated with load order.

Automatic Clustering -- the ongoing maintenance cost

For tables with a defined clustering key, Snowflake runs a background service that reorganizes micro-partitions to restore good clustering -- this consumes credits proportional to the work done.

■ EXAMPLE: Defining and inspecting a clustering key

```
-- Define a clustering key on a huge, frequently-filtered table
ALTER TABLE sales.orders
  CLUSTER BY (customer_region, order_date);

-- Check current clustering quality (lower avg_depth = better clustered)
SELECT SYSTEM$CLUSTERING_INFORMATION(
  'sales.orders',
  '(customer_region, order_date)'
);

-- See how much Automatic Clustering has cost recently
SELECT table_name, credits_used, start_time
FROM   snowflake.account_usage.automatic_clustering_history
ORDER BY start_time DESC
LIMIT 20;
```

- 1 to 3 well-chosen columns is the norm -- too many hurts more than it helps.
- Only worth doing on very large (multi-TB) tables with a clear dominant filter pattern.

Caching -- The Three Layers

A concept that spans architecture, cost, AND performance troubleshooting

---

5. Result Cache, Warehouse Cache, Metadata Cache

Result Cache (Cloud Services layer)

An exact repeat of a query, within 24 hours, on unchanged data, returns instantly with zero compute cost -- no warehouse even needs to be running.

Warehouse (Local Disk) Cache

A running warehouse caches recently-read micro-partitions on local SSD -- lost the moment the warehouse suspends.

Metadata Cache

Table/partition statistics used for pruning, and for answering trivial aggregates, sometimes without touching a running warehouse at all.

■ EXAMPLE: Seeing the Result Cache in action

```
-- First run: warehouse does real work, costs credits
SELECT COUNT(*) FROM sales.orders WHERE order_date = '2026-07-01';

-- Run the EXACT same query again within 24 hours
SELECT COUNT(*) FROM sales.orders WHERE order_date = '2026-07-01';
-- Check Query Profile: "Query Result Reuse" will show this was
-- served entirely from the Result Cache -- 0 compute credits used.

-- This DISABLES result cache for a session, useful when benchmarking:
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
```

■ *THINK OF IT LIKE THIS: Result Cache is remembering the exact answer to a question you were asked yesterday. Warehouse cache is keeping your most-used reference books open on your desk instead of walking back to the shelf.*

Time Travel & Fail-safe

Two related but distinctly different recovery mechanisms

6. Time Travel -- Self-Service Historical Access

Query, clone, or restore data exactly as it existed at a past point in time, using AT/BEFORE clauses. Default retention is 1 day (Standard), up to 90 days (Enterprise+).

Fail-safe -- the safety net beneath Time Travel

A fixed 7-day period after Time Travel ends where Snowflake can still recover data -- but only via a support request, not self-service SQL.

■ EXAMPLE: Time Travel in practice -- querying, cloning, and undoing a mistake

```
-- 1) See the table as it looked exactly 1 hour ago
SELECT * FROM sales.orders AT(OFFSET => -3600);

-- 2) See it as of a specific timestamp
SELECT * FROM sales.orders
  AT(TIMESTAMP => '2026-07-24 09:00:00'::TIMESTAMP);

-- 3) Oops -- accidentally dropped a table 20 minutes ago
DROP TABLE sales.orders;
UNDROP TABLE sales.orders;              -- instantly restored

-- 4) Oops -- accidentally ran a bad UPDATE. Recover just the old rows:
CREATE TABLE sales.orders_backup CLONE sales.orders
  BEFORE (STATEMENT => '<query_id_of_the_bad_update>');

-- 5) Set a longer retention window on a critical table (Enterprise+)
ALTER TABLE sales.orders SET DATA_RETENTION_TIME_IN_DAYS = 30;
```

Zero-Copy Cloning

Instant environments without instant cost

7. How Cloning Avoids Copying Data

CLONE creates a new object whose metadata points at the exact same underlying micro-partitions as the original -- instant, regardless of table size. Storage cost only starts once either side diverges (copy-on-write).

■ *THINK OF IT LIKE THIS: Cloning a table is like photocopying a book by handing someone the exact same book, agreeing that if either of you writes on a page, only then do you each print your own copy of just that page.*

■ **EXAMPLE: Spinning up a full dev environment from production, instantly**

```
-- Clone an entire database -- takes seconds, regardless of size
CREATE DATABASE dev_db CLONE prod_db;

-- Clone just one table, as it looked yesterday, for safe testing
CREATE TABLE orders_test
  CLONE prod_db.sales.orders
  AT(OFFSET => -86400);

-- Take a cheap "safety snapshot" right before a risky bulk update
CREATE TABLE orders_safety_snapshot CLONE sales.orders;

UPDATE sales.orders SET status = 'ARCHIVED' WHERE order_date < '2020-01-01';
-- if something goes wrong, orders_safety_snapshot is your instant rollback
```

Streams -- Change Data Capture

The mechanism behind every incremental Snowflake pipeline

8. What a Stream Tracks and How

A Stream records which rows changed (insert/update/delete) since it was last consumed, using hidden metadata columns. Querying it inside a DML transaction automatically advances its offset.

Standard vs Append-only

Standard Streams track all DML with before/after semantics. Append-only Streams only track new inserts -- simpler, for immutable event-log-style sources.

■ EXAMPLE: A full incremental merge pattern using a Stream

```
-- 1) Create a stream on the raw/Bronze source table
CREATE STREAM orders_stream ON TABLE bronze.orders;

-- 2) See only what changed since the stream was last consumed
SELECT * FROM orders_stream;
-- includes METADATA$ACTION ('INSERT'/'DELETE'), METADATA$ISUPDATE

-- 3) Merge only the changed rows into a clean Silver table
MERGE INTO silver.orders AS tgt
USING orders_stream AS src
  ON tgt.order_id = src.order_id
WHEN MATCHED AND src.METADATA$ACTION = 'DELETE' THEN DELETE
WHEN MATCHED THEN UPDATE SET
    tgt.status = src.status,
    tgt.updated_at = src.updated_at
WHEN NOT MATCHED THEN INSERT (order_id, status, updated_at)
    VALUES (src.order_id, src.status, src.updated_at);
-- Querying the stream inside this MERGE transaction automatically
-- advances its offset -- next SELECT only shows NEW changes.
```

Tasks -- Native Orchestration

How Snowflake schedules and chains work without an external tool

9. What a Task Does

A Task automates running SQL or a stored procedure, either on a schedule (cron-style) or chained AFTER another task, forming a DAG.

Conditional execution with Streams

A Task can run frequently but skip real work using WHEN SYSTEM$STREAM_HAS_DATA(...) -- avoiding cost on runs where nothing changed.

■ EXAMPLE: A 3-stage Bronze -> Silver -> Gold Task DAG

```
-- Stage 1: runs every 5 minutes, but only does work if new data exists
CREATE TASK t_bronze_to_silver
  WAREHOUSE = etl_wh
  SCHEDULE  = '5 MINUTE'
  WHEN SYSTEM$STREAM_HAS_DATA('orders_stream')
AS
  MERGE INTO silver.orders AS tgt
  USING orders_stream AS src ON tgt.order_id = src.order_id
  WHEN NOT MATCHED THEN INSERT (order_id, status)
      VALUES (src.order_id, src.status);

-- Stage 2: runs immediately AFTER stage 1 finishes (DAG dependency)
CREATE TASK t_silver_to_gold
  WAREHOUSE = etl_wh
  AFTER t_bronze_to_silver
AS
  CALL gold.refresh_daily_summary();

-- Tasks are created SUSPENDED by default -- turn them on:
ALTER TASK t_silver_to_gold RESUME;
ALTER TASK t_bronze_to_silver RESUME;
```

Snowpipe -- Continuous Ingestion

Event-driven loading vs scheduled batch loading

10. How Snowpipe Differs From a Scheduled COPY INTO

Snowpipe loads files automatically the moment they land in a stage, triggered by cloud provider event notifications (e.g. S3 event to SQS to Snowpipe), running on serverless compute -- no warehouse needed.

■ EXAMPLE: Setting up Snowpipe for auto-ingest from an S3 stage

```
-- 1) Define the external stage pointing at your S3 bucket
CREATE STAGE raw_events_stage
  URL = 's3://my-bucket/raw-events/'
  STORAGE_INTEGRATION = my_s3_integration;

-- 2) Create the pipe -- this is a saved, auto-triggered COPY INTO
CREATE PIPE bronze.events_pipe
  AUTO_INGEST = TRUE
AS
  COPY INTO bronze.events
  FROM @raw_events_stage
  FILE_FORMAT = (TYPE = 'JSON');

-- 3) Get the SQS queue ARN Snowflake generated for this pipe --
--    you configure this as the S3 bucket's event notification target
SHOW PIPES LIKE 'events_pipe';
-- look at the "notification_channel" column in the output

-- 4) Check recent load activity / troubleshoot missed files
SELECT * FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
  TABLE_NAME => 'bronze.events',
  START_TIME => DATEADD('hour', -24, CURRENT_TIMESTAMP())
));
```

Semi-Structured Data & the VARIANT Type

How Snowflake handles JSON natively without a rigid schema

11. What VARIANT Actually Stores

VARIANT holds JSON/Avro/Parquet/XML directly. Internally Snowflake analyzes structure at load time and stores it in an optimized columnar per-path form, not just raw text.

FLATTEN -- turning arrays into rows

FLATTEN explodes an array/object inside a VARIANT into multiple rows -- conceptually the same as UNNEST in other engines.

■ EXAMPLE: Querying nested JSON and flattening an array

```
-- Raw table: one row per order, 'raw_data' is a VARIANT column
-- raw_data = { "order_id": 501, "customer": {"name": "Aarav"},
--              "items": [{"sku":"A1","qty":2},{"sku":"B7","qty":1}] }

-- Access a simple nested field
SELECT raw_data:order_id::INT              AS order_id,
       raw_data:customer.name::STRING      AS customer_name
FROM   bronze.orders_raw;

-- FLATTEN the items array into one row per line item
SELECT
    raw_data:order_id::INT                        AS order_id,
    item.value:sku::STRING                        AS sku,
    item.value:qty::INT                           AS qty
FROM bronze.orders_raw,
     LATERAL FLATTEN(input => raw_data:items) AS item;
-- Output: two rows for order 501 -- one for A1, one for B7
```

Role-Based Access Control (RBAC)

The security model underlying every permission decision in Snowflake

12. Privileges Go To Roles, Never Directly To Users

You grant privileges to a Role, then grant that Role to Users (or other roles). Roles can be chained into a hierarchy.

■ *THINK OF IT LIKE THIS: Roles are keychains, not individual keys -- you hand someone the 'Warehouse Staff' keychain, not ten separate keys one by one.*

■ **EXAMPLE: Building a least-privilege custom role from scratch**

```
-- 1) Create a functional role for the data engineering team
CREATE ROLE data_engineer_role;

-- 2) Grant it exactly the privileges it needs -- nothing broader
GRANT USAGE ON WAREHOUSE etl_wh TO ROLE data_engineer_role;
GRANT USAGE ON DATABASE sales TO ROLE data_engineer_role;
GRANT USAGE ON SCHEMA sales.public TO ROLE data_engineer_role;
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA sales.public
  TO ROLE data_engineer_role;

-- 3) Put the custom role into the hierarchy under SYSADMIN
GRANT ROLE data_engineer_role TO ROLE sysadmin;

-- 4) Finally, grant it to an actual user
GRANT ROLE data_engineer_role TO USER tanu_sharma;

-- Tanu now has exactly this scoped access -- no more, no less.
```

Data Masking & Row Access Policies

Column-level and row-level security, applied dynamically

13. Dynamic Data Masking -- Column-Level Control

A masking policy shows different values for the same column depending on the querying role -- evaluated live, with no separate masked copy of the table.

Row Access Policies -- Row-Level Control

Filters which ROWS a query can see at all, enforced automatically on every query against the table -- centralizing security instead of relying on every view to add the right WHERE clause.

■ EXAMPLE: A masking policy and a row access policy, defined and applied

```
-- MASKING: hide email except for the compliance role
CREATE MASKING POLICY email_mask AS (val STRING) RETURNS STRING ->
  CASE
    WHEN CURRENT_ROLE() IN ('COMPLIANCE_ROLE') THEN val
    ELSE REGEXP_REPLACE(val, '.+\@', '***@')
  END;

ALTER TABLE sales.customers
  MODIFY COLUMN email SET MASKING POLICY email_mask;

-- ROW ACCESS: sales reps only see their own region's rows
CREATE ROW ACCESS POLICY region_policy AS (region STRING) RETURNS BOOLEAN ->
  CURRENT_ROLE() = 'COMPLIANCE_ROLE'
  OR region = CURRENT_ACCOUNT_POLICY_CONTEXT('user_region');

ALTER TABLE sales.orders
  ADD ROW ACCESS POLICY region_policy ON (customer_region);

-- Same underlying table, same SELECT * query -- different roles
-- automatically see different masked values and different row sets.
```

Secure Data Sharing

Sharing data without ever copying it

14. Live Access Instead of Data Movement

One account grants another account direct, read-only, live access to specific objects -- no export, no copy, no ETL, always current.

■ EXAMPLE: Sharing a table with a partner's Snowflake account

```
-- PROVIDER side: create a share and add objects to it
CREATE SHARE partner_share;

GRANT USAGE ON DATABASE sales TO SHARE partner_share;
GRANT USAGE ON SCHEMA sales.public TO SHARE partner_share;
GRANT SELECT ON TABLE sales.public.orders TO SHARE partner_share;

-- Add the consumer's Snowflake account to the share
ALTER SHARE partner_share ADD ACCOUNTS = ('partner_account_id');

-- CONSUMER side (in their own Snowflake account): mount the share
CREATE DATABASE partner_orders_db FROM SHARE
  provider_account_id.partner_share;

-- The consumer can now query it like any normal table --
-- data is always live, nothing was ever copied:
SELECT * FROM partner_orders_db.public.orders LIMIT 10;
```

Snowpark -- Code Where the Data Lives

The programmability layer bridging SQL and general-purpose code

15. What Problem Snowpark Solves

Instead of pulling data OUT to an external engine like Spark, Snowpark lets you write DataFrame-style Python that Snowflake pushes down and executes inside its own compute -- the code runs where the data already lives.

■ EXAMPLE: A Snowpark Python DataFrame transformation

```
from snowflake.snowpark import Session
from snowflake.snowpark.functions import col, sum as sp_sum

session = Session.builder.configs(connection_params).create()

orders_df = session.table("sales.orders")

# This looks like pandas, but it compiles down to SQL that
# executes entirely inside Snowflake's own compute -- no data
# is ever pulled out to an external Python process.
result_df = (
    orders_df
    .filter(col("order_date") >= "2026-01-01")
    .group_by("customer_region")
    .agg(sp_sum("order_total").alias("region_revenue"))
    .sort(col("region_revenue").desc())
)

result_df.write.save_as_table("gold.region_revenue_summary")
```

Materialized Views & Search Optimization

Two distinct answers to 'this specific query pattern is slow'

16. Materialized Views -- Pre-Computing Expensive Aggregations

Unlike a regular view (re-runs every time), a Materialized View physically stores the result and Snowflake keeps it incrementally in sync automatically as the base table changes.

Search Optimization Service -- Speeding Up Point Lookups

A targeted, paid feature that builds a persisted search structure to accelerate highly selective point-lookups on huge, non-clustered columns.

■ EXAMPLE: Materialized view vs Search Optimization -- two different fixes

```
-- MATERIALIZED VIEW: for a frequent, expensive single-table aggregation
CREATE MATERIALIZED VIEW gold.daily_revenue_mv AS
SELECT order_date, SUM(order_total) AS total_revenue
FROM   sales.orders
GROUP BY order_date;
-- Every dashboard query against this view is now near-instant,
-- and Snowflake keeps it updated automatically as orders change.

-- SEARCH OPTIMIZATION: for fast point-lookups on a huge table
ALTER TABLE sales.orders
  ADD SEARCH OPTIMIZATION ON EQUALITY(order_id, customer_email);

-- Now this kind of lookup on a billion-row table is fast,
-- even though order_id/email were never good clustering columns:
SELECT * FROM sales.orders WHERE order_id = 'ORD-88213411';
```

The Credit & Billing Model

Understanding this is what lets you speak credibly about cost optimization

17. Two Independent Cost Dimensions

Storage cost = compressed volume stored per month, regardless of querying. Compute cost = warehouse seconds actually run, regardless of storage volume. These are genuinely separate.

Resource Monitors -- the guardrail

Tracks credit consumption against a quota and can notify, suspend-after-current-query, or suspend-immediately when thresholds are crossed.

■ EXAMPLE: Setting up a Resource Monitor to cap runaway spend

```
CREATE RESOURCE MONITOR monthly_etl_budget
  WITH CREDIT_QUOTA = 500
  FREQUENCY = MONTHLY
  START_TIMESTAMP = IMMEDIATELY
  TRIGGERS
    ON 75  PERCENT DO NOTIFY
    ON 90  PERCENT DO SUSPEND
    ON 100 PERCENT DO SUSPEND_IMMEDIATE;

ALTER WAREHOUSE etl_wh SET RESOURCE_MONITOR = monthly_etl_budget;

-- Find your top credit-consuming warehouses this month
SELECT warehouse_name, SUM(credits_used) AS total_credits
FROM   snowflake.account_usage.warehouse_metering_history
WHERE  start_time >= DATE_TRUNC('month', CURRENT_DATE())
GROUP BY warehouse_name
ORDER BY total_credits DESC;
```

Table Types: Permanent, Transient, Temporary

A small concept that reveals real understanding of the storage model

18. Three Table Types, Three Different Data-Protection Tradeoffs

Permanent

Full Time Travel (up to 90 days) + 7-day Fail-safe. Most protection, most storage cost.

Transient

Time Travel capped at 1 day, NO Fail-safe. Cheaper -- right for regenerable ETL staging data.

Temporary

Session-scoped only, auto-dropped after, no Fail-safe. Right for scratch data within one pipeline run.

■ EXAMPLE: Choosing the right table type inside an ETL pipeline

```
-- Permanent: real production data, needs full protection
CREATE TABLE gold.customer_ltv (customer_id INT, ltv NUMBER);

-- Transient: intermediate staging data, regenerable from source,
-- doesn't need a 7-day Fail-safe safety net -- saves storage cost
CREATE TRANSIENT TABLE staging.orders_raw_today (
  order_id INT, raw_data VARIANT
);

-- Temporary: pure scratch space for one session's pipeline run
CREATE TEMPORARY TABLE tmp_dedup_check (order_id INT);
-- automatically dropped the moment this session ends
```